

Learning-based Collision Detection in Cluttered Environments

Tristan Brideweser
tbridewe@purdue.edu

Dylan Li
li4401@purdue.edu

I. PLATFORM & SIMULATION

We use the following tools and platforms:

- **Simulator:** PyBullet, which provides rigid-body physics, built-in collision detection, and straightforward scene construction.
- **Robot:** Franka Emika Panda 7-DOF manipulator, widely used in manipulation research and natively supported in PyBullet.
- **Environment:** Procedurally generated cluttered tabletop scenes. Objects of varying primitive shapes (boxes, cylinders, spheres) and YCB-style meshes are randomly placed on a table surface to create diverse clutter configurations.
- **Perception:** Synthetic depth images are rendered from a fixed overhead camera and converted to point clouds. This simulates a real-world RGB-D sensor setup.
- **Learning framework:** PyTorch is used to implement and train a PointNet-based collision classifier.
- **Motion planner:** RRT-Connect, implemented in Python, serves as the baseline planner. The geometric collision checker provided by PyBullet is used for ground-truth labels and as the baseline method.

II. APPROACH

A. Data Generation

We procedurally generate N cluttered scenes in PyBullet. For each scene, we:

- 1) Render a point cloud $\mathcal{P} \in \mathbb{R}^{M \times 3}$ from a fixed camera viewpoint.
- 2) Uniformly sample robot configurations $\mathbf{q} \in \mathbb{R}^7$ within joint limits.
- 3) Query PyBullet's built-in collision checker to obtain a binary label $y \in \{0, 1\}$ for each $(\mathcal{P}, \mathbf{q})$ pair.

We balance the dataset to ensure roughly equal representation of collision and collision-free samples, and split into training, validation, and test sets.

B. Model Architecture

The model consists of two branches:

- A **PointNet encoder** that processes the point cloud $\mathcal{P}$ into a global feature vector $\mathbf{f}_{\text{env}} \in \mathbb{R}^d$.
- A **configuration encoder** that maps the joint configuration $\mathbf{q}$ to a feature vector $\mathbf{f}_{\text{robot}} \in \mathbb{R}^d$.

The two feature vectors are concatenated and passed through a classification head to predict collision probability:

$$\hat{y} = \sigma(\text{MLP}([\mathbf{f}_{\text{env}}; \mathbf{f}_{\text{robot}}]))$$

The model is trained with binary cross-entropy loss.

C. Planner Integration

The trained classifier replaces PyBullet's geometric collision checker inside RRT-Connect. During tree expansion, each candidate configuration is evaluated by the learned model. A configuration is considered collision-free if $\hat{y} < \tau$, where τ is a tunable threshold that trades off between safety and permissiveness.

III. MILESTONES

We propose the following milestones:

- 1) **Milestone 1 (Planning/Control + Proposal):** Set up the simulated cluttered environment in PyBullet with the Franka Panda arm. Implement RRT-Connect with PyBullet's ground-truth geometric collision checker. Validate that the planner can find paths in scenes of varying clutter density. Deliver the project proposal.
- 2) **Milestone 2 (Learning Component + Initial Results):** Implement the data generation pipeline to collect $(\mathcal{P}, \mathbf{q}, y)$ tuples at scale. Train the PointNet-based collision classifier and report initial accuracy, precision, recall, and F1-score on a held-out test set. Integrate the learned model into RRT-Connect and present preliminary planning results. Deliver the project presentation.
- 3) **Milestone 3 (Full Report + Demo):** Conduct a comprehensive evaluation:
 - Compare planning time and path quality between the learned and geometric collision checkers.
 - Evaluate generalization to unseen clutter arrangements and object geometries.
 - Analyze failure modes: false negatives (unsafe paths) vs. false positives (over-conservative planning).
 - Ablate over clutter density, number of point cloud points, and classification threshold τ .

Deliver the full project report, demo videos, and documented codebase.